PvrGPU Texture Processing Unit (TPU) — Block Diagram, Data Path & Pseudocode

本文件以 block diagram、data path flow 及 pseudocode 形式詳述 `src/systemc/texture/`（包含 `texture_unit.h` 與 `texture_unit.cpp`）之 PowerVR TPU (Texture Processing Unit) 模擬架構與執行行為。

TPU 負責解析 Rogue 硬體紋理與取樣器描述符（Image/Sampler Descriptors）、計算正規化座標定址（Repeat/Clamp/MirroredRepeat）、評估 2×2 Quad 隱式導數與 Mipmap LOD、執行 Nearest / 4-tap Bilinear / 8-tap Trilinear 紋理過濾，並透過 TCU (Texture Cache Unit) / DRAM 擷取 Texel 資料交回 USC 著色器叢集。

---

0. Top-Level Architecture Block Diagram

Texture Processing Unit (TPU) — Architecture Overview
負責 USC 著色器發出的 SMP (Sample) 請求與紋理快取互動
包含雙 SC_THREAD：SampleRun (採樣過濾) 與 Run (管線同步)

↓ USC Cluster (Shader) 執行 SMP 指令 sc_port<PipelineTxn> (sample_input)

① TPU — Descriptor Decode & State Verification
- 解析 Rogue IMAGE_WORD0/1 (維度、格式、Pitch、Base Addr)
- 解析 Rogue SAMPLER_WORD0/1 (Filter、Wrap、LOD 範圍)
- 驗證 MemoryPool 內之 TextureResource 與 SamplerState

② TPU — 2×2 Quad Implicit LOD & Derivative Unit
- 輸入 2×2 Quad 4 個頂點之 UV 座標 (含 Helper Pixels)
- 計算空間偏微分：ds/dx, dt/dx, ds/dy, dt/dy
- 求解非等向性尺度 rho^2 = max(rho_x^2, rho_y^2)
- Piecewise-linear log2 近似求解連續 LOD lambda
- 決定 Mip 層級 (Level0, Level1) 及權重 (mip_weight_u8)

③ TPU — Texture Address & Filtering Core
- 座標定址：Repeat / ClampToEdge / MirroredRepeat
- 8-bit UNORM 定點數座標運算 (scaled * 256.0F, 中心 128)
- 過濾模式支援：
* Nearest (1 tap)
* Bilinear (4 taps, 單層雙線性插值)
* Trilinear (8 taps, 雙層雙線性 + Mip 權重三線性插值)

↓ 發出 Texel 讀取請求 (4 bytes per texel) sc_port<MemoryTxn> (cache_request / TCU)

④ Memory Hierarchy (TCU -> SLC -> DRAM)
- 首次採樣自動執行 Texture Preloading
- 依據計算之 Texel GPU Address 讀取 RGBA8/RGBX8 像素資料
- 回傳 MemoryTxn response

↓ 組合過濾後之 RGBA float32 採樣結果 sc_port<PipelineTxn> (sample_output)

⑤ USC Cluster (Shader Continuation)
- 接收 TextureSampleResponse
- 著色器執行 WDF (Wait For Dependency) 並合成最終顏色

---

1. Data Path Flow — 紋理採樣與管線資料流向

以下展示 TPU 處理紋理請求時，各資料結構在 USC、TPU、TCU 與 DRAM 之間的流向：

USC Cluster (Shader Execution)
1. Fragment Shader 執行 SMP (Sample) 指令
2. 組裝 TextureSampleRequest[] (含 UV 座標、Quad ID、Lane Index)
3. 填入 state.texture_sample_requests
4. 發送至 TPU sample_input (Stage: kFragmentTexturePending)

↓ sample_input

① TPU::SampleRun() — 描述符與座標解析
R: state.fragment_shared_registers -> 讀取 Image & Sampler Words
Decode: DecodeRogueTextureImageDescriptor (GPU Addr, Mips, Format)
DecodeRogueTextureSamplerDescriptor (WrapU/V, Min/Mag/Mip)
R: state.texture_resources, state.sampler_states (交叉校驗)

② TPU::SampleRun() — LOD 與定址計算
隱式 LOD (Mip-Linear):
ComputeTextureImplicitLod(2×2 coordinates) -> Level0, Level1, w8
座標定址 (Bilinear / Trilinear):
ComputeTextureLinearRepeat(u, width, wrap_u) -> x.lower, upper, wx
ComputeTextureLinearRepeat(v, height, wrap_v)-> y.lower, upper, wy

③ TPU::SampleRun() — Texel 讀取 (TCU / DRAM)
Texel Address = Image.GPU_Address + Mip.Offset + y*Pitch + x*4
IF memory_system (Direct Model):
memory_->Read(address, 4, MemoryClient::kTextureCache)
ELSE (SystemC Transaction):
cache_request->write(MemoryTxn(kRead, address, 4))
cache_response->read()

④ TPU::SampleRun() — 顏色過濾與 Lerp 運算
Nearest: 1 tap -> 直接轉換為 F32
Bilinear: 4 taps -> LerpTextureUnorm8(x) -> LerpTextureUnorm8(y)
Trilinear: 8 taps -> Sample Bilinear(Level0)
Sample Bilinear(Level1)
LerpTextureUnorm8(Level0, Level1, mip_weight8)
RGBX8 格式特別處理：強制 Alpha = 1.0F
產出 TextureSampleResponse[] (RGBA float32)
W: state.texture_sample_responses

↓ sample_output

USC Cluster (Shader Continuation)
讀取 state.texture_sample_responses，完成片段色彩輸出

↓ (主管線推進至 kFragmentShaded)

⑤ TPU::Run() — 管線同步與繞行 (Bypass) 處理
R: state.stage == kFragmentShaded
IF Non-Texture Case: 驗證無紋理請求，結算 bypass 週期
IF Texture Case: 驗證紋理計數器與流量一致性
W: state.stage = kTextureComplete
output->write(txn) -> 傳送至 PBE (Pixel Back End)

---

2. Rogue 硬體描述符解析 (Descriptor Decoding)

PowerVR Rogue 架構透過 4 個 32-bit DWORD 儲存 Image 描述符與 Sampler 描述符：

2.1 Rogue Texture Image Descriptor (`IMAGE_WORD0/1`)

```
Rogue IMAGE_WORD0 (64-bit) & STRIDE_IMAGE_WORD1 (64-bit)
IMAGE_WORD0:
[2:0] = 4 (Linear Stride 影像類型)
[7:5] = Alpha Swizzle (3: RGBA 原始 Alpha, 4: RGBX 強制 1.0)
[33:27] = 12 (U8U8U8U8 / RGBA8 UNORM 像素格式)
[47:34] = Width - 1 (14 bits, 最大 16384)
[61:48] = Height - 1 (14 bits, 最大 16384)
STRIDE_IMAGE_WORD1:
[14:0] = Stride / Pitch (以 Texel 或 Byte 為單位)
[15] = Mipmaps Present (多層 Mipmap 標記)
[53:16] = Base GPU Address (>> 2, 4-byte aligned)
[63:60] = Mip Level Count (最大 14 層)
```

2.2 Rogue Texture Sampler Descriptor (`SAMPLER_WORD0/1`)

```
Rogue SAMPLER_WORD0 (64-bit) & SAMPLER_WORD1 (64-bit)
SAMPLER_WORD0:
[12:0] = DAdjust (4095 代表 0.0 LOD Bias)
[22:13] = Min LOD (U4.6 定點數格式, 0 = 0.0)
[32:23] = Max LOD (U4.6 定點數格式, 959 = 14.984375)
[35:33] = Wrap Mode U (0: Repeat, 1: MirroredRepeat, 2: Clamp)
[37:36] = Mag Filter (0: Nearest, 1: Linear)
[39:38] = Min Filter (0: Nearest, 1: Linear)
[40] = Mip Filter (0: Nearest, 1: Linear)
[43:41] = Wrap Mode V (0: Repeat, 1: MirroredRepeat, 2: Clamp)
[49] = Normalized Coordinates (0: 正規化座標 [0,1], 1: 未正規化)
```

---

3. 座標定址與繞回模式 (Coordinate Wrapping)

TPU 支援三種標準 OpenGL ES 繞回模式，在整數與定點數空間中精準對齊：

```
Coordinate Wrapping Logic
1. Repeat (重複模式):
wrapped = ((integer % extent) + extent) % extent
將任意整數座標對齊至 [0, extent - 1] 區間
2. ClampToEdge (鉗位至邊緣):
IF integer < 0: return 0
IF integer >= extent: return extent - 1
ELSE: return integer
3. MirroredRepeat (鏡像重複):
period = extent * 2
wrapped = ((integer % period) + period) % period
IF wrapped >= extent: return (period - 1 - wrapped) (鏡像反轉)
ELSE: return wrapped
```

3.1 8-bit UNORM 線性插值定址 (`ComputeTextureLinearRepeat`)

為了重現 PowerVR 硬體行為，線性過濾運算採用 8-bit UNORM 定點數流水線：

1. 將即時浮點座標乘以 `extent * 256.0F`。
2. 進行 Nearest-Even（四捨五入偶數）取整。
3. 減去半個 Texel 中心偏移量 (`128`)。
4. 下界整數 `lower = (centered >= 0) ? centered / 256 : -((-centered + 255) / 256)`。
5. 8-bit 插值權重 `weight = centered - lower * 256`（範圍 `[0, 255]`）。
6. 上界整數 `upper = lower + 1`，並分別套用 Wrap 模式求得 Tap 索引。

---

4. 2×2 Quad 隱式 LOD 與偏微分運算

TPU 針對 2×2 Quad 的 4 個 Lane 同步計算空間偏微分，求解等向性 Mipmap 層級：

Implicit LOD Computation Flow

Step 1: 空間偏微分 (Finite Differences)
ds/dx = (UV[lane1].u - UV[lane0].u) * Image.Width
dt/dx = (UV[lane1].v - UV[lane0].v) * Image.Height
ds/dy = (UV[lane2].u - UV[lane0].u) * Image.Width
dt/dy = (UV[lane2].v - UV[lane0].v) * Image.Height

Step 2: 尺度求解 (Scale Factor Squared)
rho_x^2 = (ds/dx)^2 + (dt/dx)^2
rho_y^2 = (ds/dy)^2 + (dt/dy)^2
rho^2 = max(rho_x^2, rho_y^2)

Step 3: Piecewise-Linear Log2 近似運算
[mantissa, exponent] = frexp(rho^2)
norm_mantissa = mantissa * 2.0F
approx_log2 = (exponent - 2) + norm_mantissa
lambda = approx_log2 * 0.5F

Step 4: LOD 鉗位與 Mip 權重提取
lambda = clamp(lambda, min_lod, min(max_lod, max_mip))
Level0 = floor(lambda)
Level1 = min(Level0 + 1, max_mip)
mip_weight_u8 = clamp(floor((lambda - Level0)*256))

---

5. 紋理過濾流水線 (Filtering Datapaths)

1. Nearest Filtering (1 Texel Fetch)
Texel(x, y) 讀出 -> 直接轉換為 RGBA float32
2. Bilinear Filtering (4 Texel Fetches)
+-------------+-------------+
| Texel01 | Texel11 | -> Lerp(x.weight) -> UpperColor
+-------------+-------------+
| Texel00 | Texel10 | -> Lerp(x.weight) -> LowerColor
+-------------+-------------+
Result = Lerp(LowerColor, UpperColor, y.weight)
3. Trilinear Filtering (8 Texel Fetches)
Color0 = BilinearSample(Mip_Level0) (4 taps)
Color1 = BilinearSample(Mip_Level1) (4 taps)
Result = Lerp(Color0, Color1, mip_weight_u8)

5.1 精準定點數插值 (`LerpTextureUnorm8`)

```
FUNCTION LerpTextureUnorm8(first, second, weight):
  // 使用有號差值進行 8-bit 插值與 Round-to-Nearest-Even:
  delta = second - first
  product = weight * delta
  magnitude = abs(product)
  quotient = magnitude >> 8
  remainder = magnitude & 0xFF
  IF remainder > 128 OR (remainder == 128 AND (quotient & 1) != 0):
    quotient += 1
  result = first + (delta < 0 ? -quotient : quotient)
  RETURN clamp(result, 0, 255)
```

---

6. TextureUnit 模組內部結構

`TextureUnit` 繼承自 `sc_module`，內部包含兩個獨立運行的 `SC_THREAD`：

Thread 名稱	觸發訊號 / 埠	核心職責
`SampleRun`	`sample_input` (來自 USC)	解析描述符、計算 LOD、定址、發送 Texel 讀取至 TCU、執行 Bilinear/Trilinear 過濾、回傳結果至 `sample_output`
`Run`	`input` (主管線)	在非紋理繪製中進行 bypass 延遲計算；在紋理繪製中確認所有 sample 流量正確並推進管線至 `kTextureComplete`

---

7. 關鍵資料結構與 Handle Map

7.1 Texture 相關結構定義

結構名稱	關鍵欄位	說明
`RogueTextureImageDescriptor`	`gpu_address`, `width`, `height`, `row_pitch_bytes`, `mip_count`, `format`	解析後的 Rogue 硬體影像描述符
`RogueTextureSamplerDescriptor`	`min_filter`, `mag_filter`, `mip_filter`, `wrap_u`, `wrap_v`, `min/max_lod`	解析後的 Rogue 硬體取樣器描述符
`TextureImplicitLod`	`lambda`, `level0`, `level1`, `mip_weight_u8`, `mip_weight`	2×2 Quad 偏微分隱式 LOD 計算結果
`TextureLinearAxis`	`lower`, `upper`, `weight` (0..255)	單一座標軸之雙 Tap 索引與插值權重
`TextureSampleRequest`	`shader_lane_index`, `coordinates[2]`, `quad_id`, `quad_lane`	USC 發給 TPU 之單一 Lane 採樣請求
`TextureSampleResponse`	`shader_lane_index`, `request_id`, `rgba[4]` (float32 bits)	TPU 回傳給 USC 之單一 Lane 採樣色彩

---

8. Pseudocode 實作

8.1 隱式 LOD 計算 Pseudocode

```
FUNCTION ComputeTextureImplicitLod(coordinates[4], image, sampler):
  // 1. 計算空間偏微分
  dsdx = (coordinates[1].u - coordinates[0].u) * image.width
  dtdx = (coordinates[1].v - coordinates[0].v) * image.height
  dsdy = (coordinates[2].u - coordinates[0].u) * image.width
  dtdy = (coordinates[2].v - coordinates[0].v) * image.height

  // 2. 最大特徵尺度平方
  rho_x2 = dsdx * dsdx + dtdx * dtdx
  rho_y2 = dsdy * dsdy + dtdy * dtdy
  rho2 = max(rho_x2, rho_y2)

  // 3. Piecewise-linear log2 近似
  [mantissa, exponent] = frexp(rho2)
  approx_log2 = (exponent - 2) + (mantissa * 2.0F)
  lambda_raw = approx_log2 * 0.5F

  // 4. LOD 鉗位與 Mip 權重
  min_lod = sampler.min_lod_u4_6 / 64.0F
  max_lod = sampler.max_lod_u4_6 / 64.0F
```

```
  max_mip = image.mip_count - 1

  lambda = clamp(lambda_raw, min_lod, min(max_lod, max_mip))
  level0 = floor(lambda)
  level1 = min(level0 + 1, max_mip)

  lod_result.level0 = uint8(level0)
  lod_result.level1 = uint8(level1)
  lod_result.mip_weight_u8 = uint8(clamp(floor((lambda - level0) * 256.0F), 0, 255))
  RETURN lod_result
```

8.2 採樣與過濾核心 (`SampleRun`) Pseudocode

```
FUNCTION TextureUnit::SampleRun():
  LOOP forever:
    txn = sample_input.read()
    state = load(pool, txn.state)
    ASSERT state.stage == kFragmentTexturePending

    requests[] = load(pool, state.texture_sample_requests)
    shared_regs[] = load(pool, state.fragment_shared_registers)

    // 1. 解析描述符
    image = DecodeRogueTextureImageDescriptor(shared_regs[0..3])
    sampler = DecodeRogueTextureSamplerDescriptor(shared_regs[8..11])

    // 2. 首次採樣自動執行 Texture Preloading (若未載入)
    IF !texture_preloaded:
      PreloadTextureToDram(image.gpu_address, resource.byte_size)
      texture_preloaded = true

    // 3. 計算 Mipmap LOD (若為 Mip-Linear 模式)
    implicit_lods[] = array(size=requests.size())
    IF sampler.mip_filter == LINEAR:
      FOR quad_start = 0, step 4, quad_start < requests.size():
        quad_coords = requests[quad_start .. quad_start+3].coordinates
        lod = ComputeTextureImplicitLod(quad_coords, image, sampler)
        FOR lane IN 0..3:
          implicit_lods[quad_start + lane] = lod

    responses = []
    // 4. 逐 Request 執行採樣與過濾
    FOR index = 0 .. requests.size() - 1:
      req = requests[index]
      u = req.coordinates[0]
      v = req.coordinates[1]

      IF sampler.min_filter == NEAREST AND sampler.mag_filter == NEAREST:
        // --- Nearest Filtering ---
        tx = NearestRepeat(u, image.width, sampler.wrap_u)
        ty = NearestRepeat(v, image.height, sampler.wrap_v)
        texel = ReadTexelFromTCU(image.gpu_address, mip[0], tx, ty)
        filtered = Unorm8ToFloat(texel)

      ELSE IF sampler.mip_filter != LINEAR:
        // --- Bilinear Filtering (LOD 0) ---
        filtered = SampleBilinear(mip[0], u, v, sampler)

      ELSE:
        // --- Trilinear Filtering ---
        lod = implicit_lods[index]
        color0 = SampleBilinear(mip[lod.level0], u, v, sampler)
        color1 = SampleBilinear(mip[lod.level1], u, v, sampler)
        FOR c IN 0..3:
          filtered[c] = LerpTextureUnorm8(color0[c], color1[c], lod.mip_weight_u8) / 255.0F

      IF image.format == kRgbx8Unorm:
        filtered.a = 1.0F  // RGBX 強制 Alpha 為 1.0

      responses.append(TextureSampleResponse(req.lane_index, filtered))

    // 5. 輸出採樣回應並推進狀態
```

```
    state.texture_sample_responses = store(pool, responses)
    state.stage = kTextureSamplesReady
    wait(texture_cycles)
    store(pool, txn.state, state)
    sample_output.write(txn)
```

8.3 主管線處理 (`Run`) Pseudocode

```
FUNCTION TextureUnit::Run():
  LOOP forever:
    txn = input.read()
    state = load(pool, txn.state)
    ASSERT state.stage == kFragmentShaded

    IF UsesTextureSampling(state):
      // 驗證採樣計數器與流量匹配
      ASSERT state.counters.texture_requests > 0
      cycles = 0  // 採樣延遲已在 SampleRun 結算
    ELSE:
      // 非紋理繪製，執行 Bypass 延遲計算
      cycles = (state.active_fragment_invocations == 0) ? 0 : kReferenceUarch.texture_bypass_cycles

    state.counters.texture_cycles += cycles
    state.counters.renderer_cycles += cycles
    state.stage = kTextureComplete

    wait(cycles)
    store(pool, txn.state, state)
    output.write(txn)
```